

Assignment 1

Deadline: February 2, 23:59

Assignment instructions:

The assignment is to be done in **pairs**. You'll need to join a **group on Canvas** to be able to submit a solution for grading.

Individual or late submissions will not be graded.

Your submission will consist of the following components:

1. A **PDF-report**. You are to write your report in $\text{\LaTeX}$. You may use the provided template on Canvas if you wish.
2. Your **code**. This can be included in **either** an appendix, or a standalone file-upload. Please do not upload a zip-file containing multiple files, rather upload multiple `.py` or `.ipynb` files.

Please read the “General assignment information” before starting to work on the assignment, available in the module “Assignment Instructions” on Canvas.

In case you run into problems, the consultation hours are on Wednesdays 13:15-15:00.

You are also free to reach out with questions or swing by our office whenever, if we're available we'll help.

Best of luck! - TAs

Introduction and available tools

Throughout this course you will work with graph-structured data using PyTorch Geometric (PyG).

You are expected to actively consult the official documentation of the libraries used. In particular, refer to the PyTorch Geometric documentation:

<https://pytorch-geometric.readthedocs.io/en/latest/>

You do not need to understand the full library. Focus on how datasets are loaded, how graphs are stored, and how to access node- and edge-level information.

The same expectation applies to the following libraries:

- **NetworkX**, used primarily for graph handling and visualization: <https://networkx.org/en/>
- **PyTorch**, the underlying machine learning framework for defining and training models: <https://docs.pytorch.org/tutorials/beginner/basics/intro.html>

In addition, standard Python libraries such as **numpy**, **scikit-learn**, and **matplotlib** will be useful for numerical computations, simple models, and visualization. You are expected to look up relevant functionality in the respective documentation when needed.

1. Node Embeddings with PyTorch Geometric (PyG) [2 pts]

Goal: In this assignment you will learn how to work with graph data using PyTorch Geometric (PyG). The focus is on understanding graph data structures, creating node embeddings from both graph structure and node features, visualizing them, and using them for node classification.

Dataset and setup

Use the **Karate Club** graph dataset from `torch_geometric.datasets.KarateClub`.

The dataset contains:

- a single graph,
- nodes representing members of a karate club,
- edges representing social interactions,
- node labels indicating the final split of the club.

Part A: PyG basics

1. Load the Karate Club dataset using PyTorch Geometric.
2. Print and report:
 - number of nodes,
 - number of edges,
 - shape of `data.x` (if available),
 - shape of `data.edge_index`,
 - number of node labels/classes.
3. In your own words (2–3 sentences), explain the role of:

`data.x`, `data.edge_index`, `data.y`.

Part B: Visualizing the graph

1. Convert the PyG graph to a **NetworkX** graph.
2. Visualize the graph using any layout of your choice.
3. Color nodes according to their labels.

Part C: Structural feature embeddings

In this part you will build node embeddings using **only graph structure**. You must compute **all** the following structural features for each node:

- Degree,
- Betweenness centrality,
- Closeness centrality,
- PageRank,
- Eigenvector centrality,
- Clustering coefficient.

C1. Implement structural features

1. Write a function of your own to implement the and compute the structural features.
2. Convert each feature into a vector aligned with node indices.
3. Stack all features to form a structural embedding matrix:

$$X_{\text{struct}} \in \mathbb{R}^{N \times d_s}.$$

Important: You must implement the full pipeline yourself:

- computing features,
- handling node ordering,
- stacking them into a matrix,
- normalizing the features.

Do **not** use any ready-made embedding function that directly outputs node embeddings.

C2. Visualize structural embeddings

1. Normalize the structural features.
2. Reduce them to 2D using PCA.
3. Plot the 2D embeddings and color nodes by label.

C3. Node classification using structural features

1. Train a simple classifier (in **PyTorch**) (e.g., logistic regression or a small MLP) using X_{struct} .
2. Report the classification accuracy.

Part D: Node classification using node features only

1. Use `data.x` (node features only) to train the same classifier.
2. Report the classification accuracy.
3. Visualize the node features in 2D using PCA and color by label.

Part E: Combining structure and node features

1. Concatenate structural features and node features:

$$X_{\text{comb}} = \begin{bmatrix} X_{\text{struct}} & X_{\text{attr}} \end{bmatrix}.$$

2. Train the classifier on X_{comb} .
3. Report the classification accuracy.

Part F: Repeat the classification on Cora (Planetoid)

In this part you will repeat **the classification experiments** on the Cora dataset.

Dataset. Use the Cora dataset from `torch_geometric.datasets.Planetoid`.

Task. Repeat the node label prediction using the same three inputs as before:

1. Structural features only (X_{struct}),
2. Node features only ($X_{\text{attr}} = \text{data.x}$),
3. Combined features ($X_{\text{comb}} = \begin{bmatrix} X_{\text{struct}} & X_{\text{attr}} \end{bmatrix}$).

Important notes.

- Use the official masks provided by PyG:

`train_mask, val_mask, test_mask.`

- Use a classifier such as logistic regression or a small MLP to train and predict the labels.
- Report both validation accuracy and test accuracy for each of the three settings.
- if some structural features can be expensive on larger graphs, you may use an approximation for betweenness centrality (if needed). If you do, clearly explain what you did.

Part G: Short discussion

Answer briefly:

1. Which input performed best: structural features, node features, or both?
2. Why do you think combining them helps or does not help?
3. What kind of information is captured by structural features but not by node features?

2. Node2Vec vs Struct2Vec on GeometricShapes [4 pts]

In this assignment you will implement and compare two classic random-walk-based node embedding methods:

- **Node2Vec** (proximity-based embeddings),
- **Struct2Vec** (structure-based embeddings).

The goal is to understand *what kind of similarity* each method captures and how this affects the learned node embeddings.

Details regarding implementation and theory of Node2Vec and Struct2Vec are found in:

- A. Grover and J. Leskovec, *node2vec: Scalable Feature Learning for Networks*, KDD 2016.
<https://arxiv.org/pdf/1607.00653>
- L. R. A. Ribeiro et al., *struc2vec: Learning Node Representations from Structural Identity*, KDD 2017.
<https://arxiv.org/pdf/1704.03165>

Dataset

Use the `GeometricShapes` dataset from `torch_geometric.datasets.GeometricShapes`. The dataset contains **40 graphs**, each representing a geometric shape. Each graph provides node coordinates `data.pos` that give the 2D or 3D positions of nodes.

Important note about edges. In `GeometricShapes`, the mesh connectivity is stored as `data.face` (mesh faces), not as `edge_index`. Convert the mesh into a graph by using `torch_geometric.transforms.FaceToEdge` as a pre-transform.

Task overview

For each selected graph you will:

1. Plot the **original graph** using `data.pos` and edges.
2. Compute node embeddings using:
 - your **Node2Vec** implementation,
 - your **Struct2Vec** implementation (use $k = 3$ and your defined $g_k(u, v)$).
3. Plot the graph again, but now place nodes using the learned embeddings (2D), and compare them to the original geometry.

Part A: Load graphs and visualize the original geometry

1. Load `GeometricShapes` with `FaceToEdge` so that each `Data` object has `edge_index`.
2. Select **at least 3 graphs** from the dataset (any indices you like).
3. For each selected graph:
 - Plot the graph using `data.pos` as node coordinates.
 - Draw edges using `data.edge_index`.

Part B: Node2Vec embeddings and visualization

B1. Implement biased random walks. Implement Node2Vec random walks on an undirected graph $\mathcal{G} = (V, E)$.

Given the previous node t and current node v , define the transition probability to a neighbor $x \in \mathcal{N}(v)$ as:

$$\pi_{vx} \propto \alpha_{pq}(t, x),$$

where

$$\alpha_{pq}(t, x) = \begin{cases} \frac{1}{p} & \text{if } x = t, \\ 1 & \text{if } (t, x) \in E, \\ \frac{1}{q} & \text{otherwise.} \end{cases}$$

Here, $(t, x) \in E$ means that there is an edge connecting t and x in the graph.

You must report the parameters p and q used in your implementation.

B2. Generate training walks. For each node:

- generate multiple random walks,
- use a fixed walk length (e.g., 20).

B3. Learn embeddings. Train node embeddings using the skip-gram model with negative sampling.

For each selected graph:

1. Run your **Node2Vec** implementation on the graph and produce an embedding matrix $Z \in \mathbb{R}^{n \times d}$. Use $d = 2$ so that embeddings can be plotted directly.
2. Run Node2Vec with at least two different settings:
 - BFS-like walks (small q),

- DFS-like walks (large q).

3. Plot the graph again, but now use Z as node coordinates (and draw the same edges).

Part C: Struct2Vec embeddings and visualization (fixed $k = 3$)

Struct2Vec is designed to capture **structural equivalence** rather than proximity.

C1. Structural similarity function $g_k(u, v)$. Fix $k = 3$.

For each node u , define its structural signature as the multiset of node degrees in its k -hop neighborhood:

$$S_k(u) = \{ \deg(w) \mid \text{dist}(u, w) \leq k \}.$$

Define the structural similarity between two nodes u and v as:

$$g_k(u, v) = \exp \left(-\frac{1}{|S_k(u)|} \sum_{i=1}^{|S_k(u)|} |S_k^{(i)}(u) - S_k^{(i)}(v)| \right),$$

where $S_k^{(i)}(\cdot)$ denotes the sorted degree sequence.

C2. Construct the Struct2Vec graph. Construct a **complete weighted graph** where:

- each node corresponds to a node in the original graph,
- edge weight between u and v is $g_k(u, v)$.

Note: If the complete graph is too large, you may use a sparse approximation (e.g., connect each node to its top- m most similar nodes). Clearly state your choice.

C3. Random walks on the Struct2Vec graph. Perform random walks on the weighted Struct2Vec graph:

- transition probabilities must be proportional to edge weights,
- unlike Node2Vec, walks are **not restricted** to original neighbors.

C4. Learn embeddings. For each selected graph:

1. Compute structural similarity using your fixed definition of $g_k(u, v)$ with $k = 3$.
2. Generate random walks and train embeddings $Z_{\text{struct}} \in \mathbb{R}^{n \times d}$ with $d = 2$.
3. Plot the graph using Z_{struct} as node coordinates.

Part D: Comparison questions (short answers)

Answer briefly (2–4 sentences each):

1. Compare the three plots for each graph: **(i)** original geometry (**pos**), **(ii)** Node2Vec embedding plot, **(iii)** Struct2Vec embedding plot. Which one preserves the geometric shape best? Why?
2. What type of similarity does Node2Vec mainly capture?
3. Why does Node2Vec struggle to capture structural similarity?
4. What kind of nodes benefit most from Struct2Vec?

3. Graph-Based Sentiment Analysis with Node2Vec [4 pts]

Goal: In this task you will build a **word graph** from text data, learn **word embeddings** using **Node2Vec**, and use these embeddings to perform **sentiment classification**.

This assignment demonstrates how graph-based embeddings can be used beyond classical network data and applied to natural language processing tasks.

Dataset

Use a sentiment analysis dataset consisting of short texts and binary labels (positive / negative). You may use one of the following datasets:

- **IMDb movie reviews dataset:**
<https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>
- **Twitter sentiment dataset:**
<https://www.kaggle.com/datasets/saurabhshahane/twitter-sentiment-dataset>

You may use the provided train/test split or create your own split.

Part A: Construct a word co-occurrence graph

A1. Text preprocessing. Preprocess the text corpus:

- convert all text to lowercase,
- remove punctuation and special characters,
- tokenize text into words,
- optionally remove stop words.

A2. Build the word graph. Construct an undirected word co-occurrence graph:

- each node corresponds to a unique word (you may limit the vocabulary size to reduce computation),
- add an edge between two words if they appear within a fixed context window of size w in the same text (e.g., $w = 2$ or $w = 3$),
- optionally weight edges by co-occurrence frequency.

Briefly explain your design choices (window size, vocabulary size, edge weighting).

Part B: Learn word embeddings using Node2Vec

B1. Apply Node2Vec. Using your own implementation of Node2Vec from the previous assignment:

- generate random walks on the word graph,
- train word embeddings using the skip-gram model with negative sampling.

Choose and report reasonable values for:

- embedding dimension,
- walk length,
- number of walks per node,
- Node2Vec parameters p and q .

B2. Inspect embeddings. Visualize a subset of word embeddings using t-SNE and briefly comment on whether words with similar sentiment appear close to each other.

Part C: Text representation using graph embeddings

C1. Document embeddings. Represent each text/document using the learned word embeddings. You may use one of the following:

- average of word embeddings,
- weighted average using word frequency or TF-IDF,
- a recurrent neural network (RNN) over word embeddings,
- another aggregation method (clearly explain it).

This results in one embedding vector per document.

Part D: Sentiment classification

D1. Train a classifier. Using the document embeddings:

- train a classifier (e.g., logistic regression or a small MLP),
- evaluate sentiment classification accuracy on the test set.

Part E: Short discussion

Answer briefly (2–4 sentences each):

1. What information does the word co-occurrence graph capture that bag-of-words does not?
2. How do the Node2Vec parameters p and q affect the learned word embeddings?
3. What are the main advantages and disadvantages of this graph-based approach compared to standard NLP features (e.g., TF-IDF)?